

Lab3

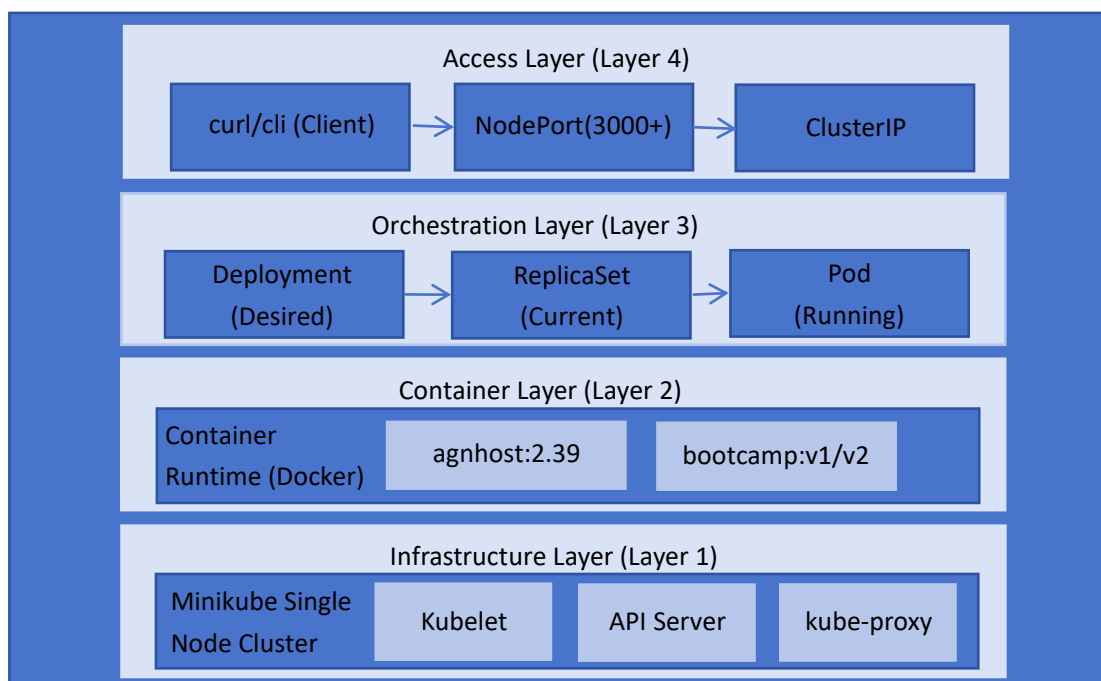
name: WangShiyu nuist_id: 202383910020 setu_id: W20109917

1. Experiment Purpose

The primary objective of this experiment is to establish a comprehensive understanding of container orchestration fundamentals through hands-on interaction with Kubernetes.

- 1) **Master Cluster Environment Setup:** Learn to deploy a local Kubernetes cluster using Minikube and verify the operation of essential toolchains (Docker, kubectl) and cluster components.
- 2) **Understand Core Objects:** Grasp the concepts and hierarchy of core Kubernetes primitives, such as Pods, Deployments, and Services, and understand how Labels and Selectors enable loose coupling.
- 3) **Practice Application Management:** Develop proficiency with the kubectl CLI to perform essential operations and understand Declarative State Management where Kubernetes maintains the desired state automatically.
- 4) **Master Service Networking:** Comprehend different Service types and gain practical experience exposing applications using NodePort, understanding traffic flow and load balancing.
- 5) **Verify Elastic Scaling:** Learn to scale applications horizontally by adjusting replica counts and observe how the Scheduler and Services manage Pod distribution and traffic load balancing.
- 6) **Manage Lifecycle and Updates:** Implement Rolling Update strategies for zero-downtime deployment and practice rollback procedures to ensure service reliability and rapid recovery.
- 7) **Cultivate Engineering Mindset:** Learn resource cleanup best practices and develop troubleshooting skills using diagnostic tools like kubectl describe and kubectl logs.

2. Experiment Process



In this experiment, I used Minikube to build a local Kubernetes cluster and completed the official Kubernetes beginner tutorials, including application deployment, exploration, service exposure, scaling, updating, and rollback.

1) Environment Preparation

Before starting the experiment, I checked whether the required tools had been installed correctly, including Docker, kubectl and Minikube. These tools are necessary because Docker provides the container runtime environment, Minikube provides a local Kubernetes cluster, and kubectl is used to interact with the Kubernetes API server.

(1) Check the installation status

```
PS C:\Users\WangShiyu> kubectl version --client
Client Version: v1.28.3
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
PS C:\Users\WangShiyu> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

(2) Start the Minikube cluster with the Docker driver.

```
PS C:\Users\WangShiyu> minikube start
* Microsoft Windows 11 Home China 25H2 上的 minikube v1.38.1
* 自动选择 docker 驱动
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* 使用具有 root 权限的 Docker Desktop 驱动程序
* 在集群中 "minikube" 启动节点 "minikube" primary control-plane
* 正在拉取基础镜像 v0.0.50 ...
! minikube cannot pull kicbase image from any docker registry, and is trying to download kicbase tarball from github release page via HTTP.
! 您很有可能遇到了网络问题。请确保您至少可以通过 HTTP（直连或代理）访问互联网。当前您的代理配置为：
> kicbase-v0.0.50-amd64.tar: 232.29 MiB / 1.28 GiB 17.69% 50.24 KiB p/s 1
E0507 16:55:03.014067 32332 cache.go:239] Error downloading kic artifacts: failed to download kic base image or any fallback image
* 创建 docker container (CPU=2, 内存=4000MB) ...|
* 正在 Docker 29.2.1 中准备 Kubernetes v1.35.1-
* 配置 bridge CNI (Container Networking Interface) ...
* 正在验证 Kubernetes 组件...
- 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
* 启用插件: storage-provisioner, default-storageclass

! C:\kubectl\kubectl.exe 的版本为 1.28.3, 可能与 Kubernetes 1.35.1 不兼容。
- 想要使用 kubectl v1.35.1 吗? 尝试使用 'minikube kubectl -- get pods -A' 命令
* 完成! kubectl 现在已配置, 默认使用 "minikube"集群和"default"命名空间
```

(3) Check the cluster status

The output showed that the host, kubelet, API server, and kubeconfig were all running or configured correctly. This indicated that the Kubernetes cluster had been started successfully.

```
PS C:\Users\WangShiyu> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Component	Status	Description	Explanation
host	Running	Minikube host machine	Represents the Minikube virtual machine or container that hosts the Kubernetes cluster
kubelet	Running	Node agent managing pods	A critical agent running on each node, responsible for managing the lifecycle of Pods and ensuring containers are running correctly
apiserver	Running	Kubernetes API server	The central management component that exposes the Kubernetes API, serving as the frontend for the Kubernetes control plane

kubeconfig	Configured	kubectl configuration	Contains authentication information and cluster connection details, enabling kubectl to communicate with the cluster
------------	------------	-----------------------	--

(4) Check the Kubernetes node

The node named minikube was shown as “Ready”, which means that the Kubernetes cluster was ready to run workloads.

```
PS C:\Users\WangShiyu> kubectl get nodes
NAME      STATUS   ROLES    AGE   VERSION
minikube   Ready    control-plane  4h18m  v1.35.1
```

In production environments, clusters typically have multiple nodes for high availability and scalability. The Ready status indicates that the node is healthy and has sufficient resources to accept new Pods.

- In Kubernetes, nodes are worker machines where containers are deployed.
- Minikube creates a single-node cluster where the control plane and workloads run on the same node.

2) Hello Minikube

In this part, I completed the official Hello Minikube tutorial. The purpose of this step was to create a simple application in the Kubernetes cluster and expose it through a Service.

(1) Create a Deployment named hello-minikube

A Deployment is a Kubernetes resource used to manage application replicas. After the Deployment was created, Kubernetes automatically created a Pod to run the container.

There are some problems while doing this step, but I figured out the errors by studying and successfully resolved them!

a) Problem Description

When executing the Hello Minikube module, I ran the following command as required by the tutorial to create a Deployment: kubectl create deployment hello-node

--image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080

After executing the command, the Pod remained in ImagePullBackOff status for an extended period and could not run properly.

b) Root Cause Analysis

After troubleshooting, I found that the root cause of the problem was network access restrictions.

- registry.k8s.io is the official Kubernetes image registry, with servers located abroad
- In my network environment, accessing this image registry frequently resulted in connection timeouts, extremely slow speeds, or complete inaccessibility
- Kubernetes could not successfully pull the required container image, causing the Pod to fail to start

c) Solution

Step 1: Load Local Image to Minikube

Use the minikube image load command to load an existing local Docker image into Minikube's

internal container runtime.

This command imports the httpd:2.4 image from the local Docker environment into the Minikube cluster, allowing Kubernetes to use the local image directly without pulling from the internet.

Step 2: Create Deployment Using Local Image

Create a Deployment using the loaded httpd:2.4 image

```
PS C:\Users\WangShiyu> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	IN USE
docker.gitea.com/gitea:1.26.1	d8667667b4cc	245MB	69.3MB	U
docker/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U
gcr.io/k8s-minikube/kicbase@sha256:eb4fec00e8ad70adf8e6436f195cc429825ffb85f95afcd5d8d9deb576f3e93	eb4fec00e8ad	1.94GB	545MB	U
httpd:2.4	89a0b59e6d72	177MB	47.6MB	
jocatalin/kubernetes-bootcamp:v2	fb1a3ced00ce	327MB	83.6MB	
mysql:8.0	d0304ed9fdb6	1.1GB	249MB	U
nginx:latest	97cfd6bc0038	237MB	63MB	
node:22-alpine	8ea2348b068a	230MB	57.5MB	U
postgres:latest	78481659c47e	649MB	168MB	
redis:latest	832d7785830f	204MB	55.3MB	
web:latest	9de5aff8ed33	1.59GB	402MB	

```
PS C:\Users\WangShiyu> minikube image load httpd:2.4
PS C:\Users\WangShiyu> minikube image ls | findstr httpd
```

It can be seen that both the Deployment and Pod were successfully created and are running normally.

(2) Check the Deployment

```
PS C:\Users\WangShiyu> kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-minikube	1/1	1	1	9s

(3) Check the Pod status

After waiting for a short time, the Pod status became Running, which means that the application container was successfully started.

```
PS C:\Users\WangShiyu> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
hello-minikube-75b65c4879-5wnjt	1/1	Running	0	15s

(4) Expose the Deployment using a NodePort Service

```
PS C:\Users\WangShiyu> kubectl expose deployment hello-minikube --type=NodePort --port=80
service/hello-minikube exposed
```

- kubectl expose: creates a Service resource that provides network access to a set of Pods
- --type=NodePort: exposes the Service on each node's IP address at a static port (NodePort), making the application accessible from outside the cluster
- --port=80: specifies the Service port that maps to the target port on the Pods

(5) Check the Service

The output showed that a Service named hello-minikube had been created. Since the Service type was NodePort, the application could be accessed from outside the cluster through a node port.

```
PS C:\Users\WangShiyu> kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hello-minikube	NodePort	10.98.8.243	<none>	80:32436/TCP	7s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4h7m

(6) Access the Service

Because the experiment was performed in a local Minikube environment, I used the following command to get the access URL.

```
PS C:\Users\WangShiyu> minikube service hello-minikube --url
http://127.0.0.1:53358
```

Then I used curl to access the application.

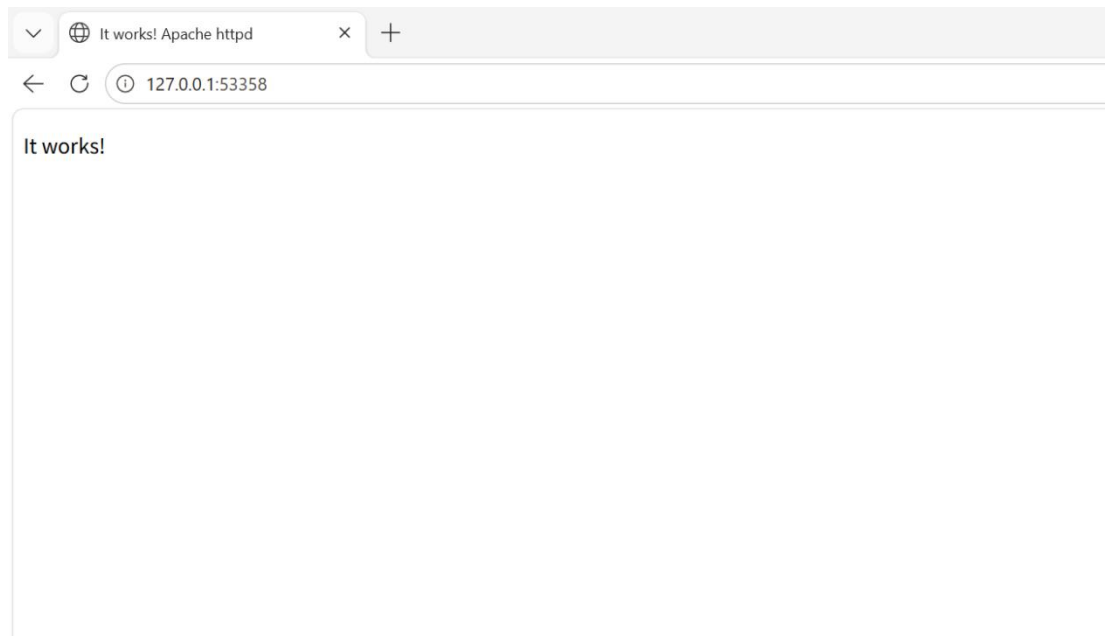
```
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:53358
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
```

The curl command sends an HTTP GET request to the application endpoint. A successful response indicates that the entire request flow is working correctly:

- The Service routes traffic to the correct Pod
- The Pod is running the container successfully
- The container is listening on the expected port

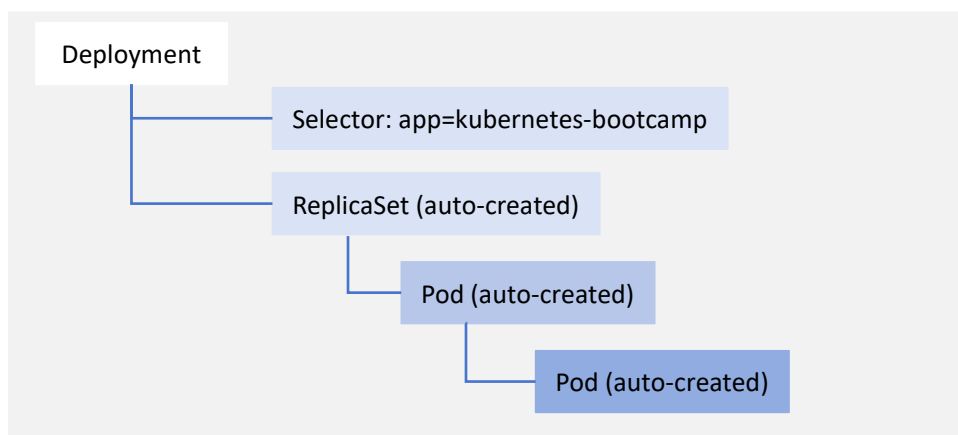
This end-to-end validation confirms that all Kubernetes components are functioning properly.

Besides, I used browser to access the application.



The successful response proved that the hello-minikube application was running and accessible.

3) Deploy an App



- Deployment: The top-level controller that defines the desired state, including the Pod template and replica count
- ReplicaSet: Automatically created and managed by Deployment, responsible for maintaining the specified number of Pod replicas
- Pod: Contains one or more containers that run the application
- Selector: Uses labels to identify which Pods belong to the Deployment

In this part, I followed the Kubernetes Basics tutorial to deploy the sample application `kubernetes-bootcamp`.

(1) Create a Deployment named `kubernetes-bootcamp`

This command created a Deployment using the image `gcr.io/google-samples/kubernetes-bootcamp:v1`. Kubernetes then created the corresponding ReplicaSet and Pod automatically.

```
PS C:\Users\WangShiyu> kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1
deployment.apps/kubernetes-bootcamp created
```

Unlike the previous `hello-minikube` example, this image uses the default entrypoint, so no additional command arguments are needed.

- `kubernetes-bootcamp:v1`: is a simple web application that responds with a greeting message and version information
- `:v1`: specifies the version of the image, which is important for the update experiment later

(2) Check the Deployment

```
PS C:\Users\WangShiyu> kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
hello-minikube 1/1     1            1           35m
kubernetes-bootcamp 1/1     1            1           36s
```

(3) Check the Pod

The Pod status became Running, which indicated that the application had been deployed successfully.

```
PS C:\Users\WangShiyu> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
hello-minikube-75b65c4879-5wnjt     1/1     Running   0           35m
kubernetes-bootcamp-67fbdd6b79-m5fnx 1/1     Running   0           44s
```

4) Explore Your App

In this part, I explored the running application by checking Pod details, viewing logs, and executing commands inside the container.

(1) Store the Pod name in an environment variable

Since I used PowerShell in Windows, I used the following command, and then I printed the variable.

```
PS C:\Users\WangShiyu> $POD_NAME = kubectl get pods -l app=kubernetes-bootcamp -o jsonpath="{.items[0].metadata.name}"
PS C:\Users\WangShiyu> echo $POD_NAME
kubernetes-bootcamp-67fbdd6b79-m5fnx
```

- The `kubectl get pods` command with the `-o jsonpath` option extracts specific fields from the output using JSONPath syntax.
- `{.items[0].metadata.name}` retrieves the name of the first Pod in the list.

- ◆ .items: Array of Pod objects in the list
- ◆ [0]: Index of the first Pod in the array
- ◆ .metadata: Metadata field of the Pod object
- ◆ .name: Name field within metadata

This technique is useful for automation and scripting, as it allows you to dynamically obtain Pod names for use in subsequent commands. Besides, Pod names are dynamically generated by Kubernetes based on the Deployment name and a random suffix, so they cannot be predicted in advance.

(2) View the detailed information of the Pod

The output contained important information such as Pod status, node name, IP address, container image, container state, and events. This helped me understand how Kubernetes schedules and runs Pods.

```
PS C:\Users\WangShiyu> kubectl describe pods $POD_NAME
Name:          kubernetescbootcamp-67fbd6b79-m5fnx
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Thu, 07 May 2026 22:09:02 +0800
Labels:        app=kubernetescbootcamp
               pod-template-hash=67fbd6b79
Annotations:   <none>
Status:        Running
IP:            10.244.0.17
IPs:
  IP:          10.244.0.17
Controlled By: ReplicaSet/kubernetescbootcamp-67fbd6b79
Containers:
  kubernetescbootcamp:
    Container ID:  docker://ad6ff119592a86775b4fdc890b205f6f81d4b5d1c21d856d08224ff58c667766
    Image:         gcr.io/google-samples/kubernetescbootcamp:v1
    Image ID:      docker-pullable://gcr.io/google-samples/kubernetescbootcamp@sha256:0d6b8ee63bb57c5f5b6156f446b3bc3b3c143d233037f3a2f00e279c8fcc64af
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Thu, 07 May 2026 22:09:27 +0800
    Ready:         True
    Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-qp2rg (ro)
Conditions:
  Type                 Status
  PodReadyToStartContainers  True
  Initialized            True
  Ready                  True
  ContainersReady        True
  PodScheduled           True
Volumes:
  kube-api-access-qp2rg:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:       kube-root-ca.crt
    ConfigMapOptional:   <nil>
    DownwardAPI:         true
QoS Class:              BestEffort
Node-Selectors:          <none>
Tolerations:             node.kubernetesc.io/not-ready:NoExecute op=Exists for 300s
                        node.kubernetesc.io/unreachable:NoExecute op=Exists for 300s
Events:
  <none>
```

- describe: Shows the complete state of the Pod, including its current phase, assigned node, IP address, container statuses, and events

(3) check the application logs

The logs showed the runtime output of the application container.

```
PS C:\Users\WangShiyu> kubectl logs $POD_NAME
Kubernetesc Bootcamp App Started At: 2026-05-07T14:09:27.532Z | Running On: kubernetescbootcamp-67fbd6b79-m5fnx

Running On: kubernetescbootcamp-67fbd6b79-m5fnx | Total Requests: 1 | App Uptime: 59523.477 seconds | Log Time: 2026-05-08T06:41:31.010Z
Running On: kubernetescbootcamp-67fbd6b79-m5fnx | Total Requests: 2 | App Uptime: 59524.002 seconds | Log Time: 2026-05-08T06:41:31.534Z
Running On: kubernetescbootcamp-67fbd6b79-m5fnx | Total Requests: 3 | App Uptime: 59906.753 seconds | Log Time: 2026-05-08T06:47:54.285Z
```

- logs: Retrieves standard output and error streams from the container, essential for troubleshooting application issues

(4) Execute the command inside the Pod

This command printed the environment variables inside the container, which showed that Kubernetes injects service-related environment variables into Pods.

```
PS C:\Users\WangShiyu> kubectl exec $POD_NAME -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=kubernetes-bootcamp-67fbd6b79-m5fnx
HELLO_MINIKUBE_PORT_80_TCP_PROTO=tcp
HELLO_MINIKUBE_PORT_80_TCP_PORT=80
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
HELLO_MINIKUBE_PORT_80_TCP=tcp://10.100.139.56:80
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HELLO_MINIKUBE_SERVICE_PORT=80
HELLO_MINIKUBE_PORT=tcp://10.100.139.56:80
HELLO_MINIKUBE_PORT_80_TCP_ADDR=10.100.139.56
KUBERNETES_SERVICE_HOST=10.96.0.1
HELLO_MINIKUBE_SERVICE_HOST=10.100.139.56
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
NPM_CONFIG_LOGLEVEL=info
NODE_VERSION=6.3.1
HOME=/root
```

- exec: Allows running arbitrary commands inside a running container, similar to docker exec

(5) Enter the container shell

Inside the container, I accessed the application locally.

The output showed that the application responded successfully. This confirmed that the application was running inside the container and listening on port 8080.

```
PS C:\Users\WangShiyu> kubectl exec -ti $POD_NAME -- bash
root@kubernetes-bootcamp-67fbd6b79-m5fnx:/# curl http://localhost:8080
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbd6b79-m5fnx | v=1
root@kubernetes-bootcamp-67fbd6b79-m5fnx:/# exit
exit
```

- -ti: Allocate a pseudo-TTY and keep stdin open, enabling interactive sessions

Finish all of these, exited the container.

5) Expose Your App Publicly

In this part, I exposed the kubernetes-bootcamp application so that it could be accessed from outside the cluster.

(1) Expose the kubernetes-bootcamp Deployment with a NodePort Service

```
PS C:\Users\WangShiyu> kubectl expose deployment/kubernetes-bootcamp --type="NodePort" --port 8080
service/kubernetes-bootcamp exposed
```

- kubectl expose: Creates a Service that targets Pods matching the Deployment's selector labels
- --type="NodePort": Instructs Kubernetes to allocate a port from the configured NodePort range

(2) Check the Services

A new Service named kubernetes-bootcamp was created.

```
PS C:\Users\WangShiyu> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
hello-minikube      NodePort    10.100.139.56   <none>       80:30523/TCP     16h
kubernetes           ClusterIP   10.96.0.1       <none>       443/TCP          20h
kubernetes-bootcamp NodePort    10.101.102.184 <none>       8080:32201/TCP   9s
```

(3) View the detailed information of the Service

The output showed the Service type, ClusterIP, NodePort, endpoints, labels, and selectors. The selector `app=kubernetes-bootcamp` allowed the Service to forward traffic to the corresponding Pods.

```
PS C:\Users\WangShiyu> kubectl describe services/kubernetes-bootcamp
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
Name:                kubernetes-bootcamp
Namespace:            default
Labels:               app=kubernetes-bootcamp
Annotations:          <none>
Selector:             app=kubernetes-bootcamp
Type:                NodePort
IP Family Policy:     SingleStack
IP Families:          IPv4
IP:                  10.101.102.184
IPs:                 10.101.102.184
Port:                <unset> 8080/TCP
TargetPort:           8080/TCP
NodePort:             <unset> 32201/TCP
Endpoints:            10.244.0.17:8080
Session Affinity:     None
External Traffic Policy: Cluster
Events:               <none>
```

(4) Obtain the NodePort value

```
PS C:\Users\WangShiyu> $NODE_PORT = kubectl get services/kubernetes-bootcamp -o jsonpath="{.spec.ports[0].nodePort}"
PS C:\Users\WangShiyu> echo $NODE_PORT
32201
```

Normally, the application can be accessed through: `curl http://$(minikube ip):$NODE_PORT`. However, because my environment used the Windows Docker driver for Minikube, direct access through minikube ip and NodePort could time out. Therefore, I used the Minikube service tunnel method.

```
PS C:\Users\WangShiyu> minikube service kubernetes-bootcamp --url
http://127.0.0.1:57354
```

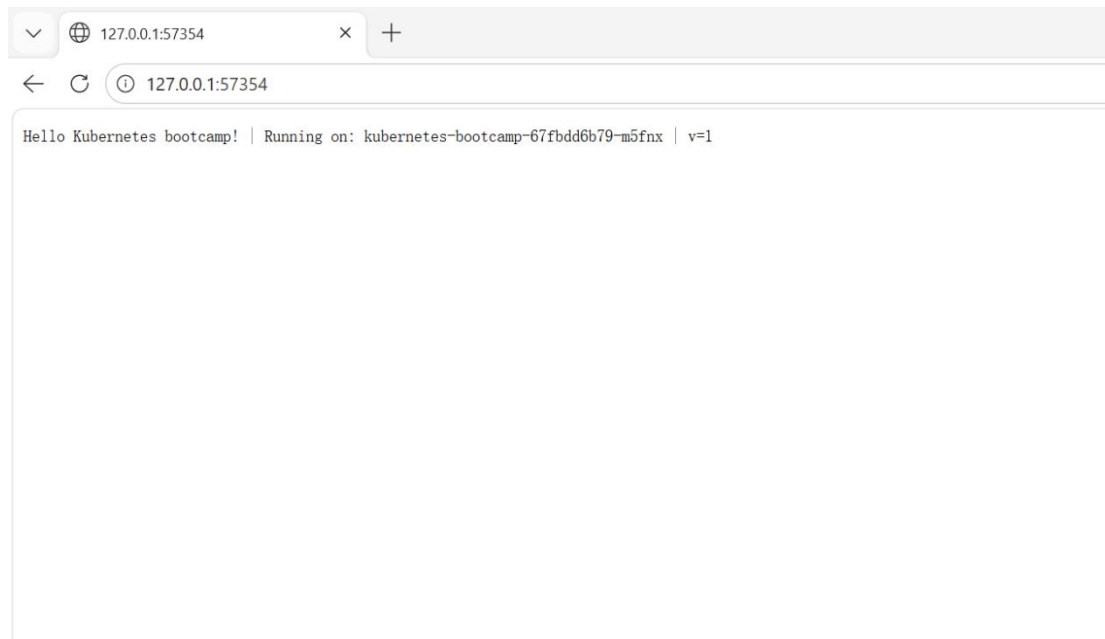
- minikube service: Creates a network tunnel from the host to the Minikube container, mapping a local port to the Service
- --url: Outputs the tunnel URL without opening a browser

(5) Use curl and browser to access the application

The response showed the Hello Kubernetes bootcamp message, which proved that the application had been exposed successfully.

```
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:57354
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbd6b79-m5fnx | v=1
```

- curl.exe (note the .exe extension on Windows to distinguish from PowerShell alias): sends a request through this tunnel



(6) Delete the Service

Finally, according to the official tutorial, I deleted the Service, and checked the Services and Pods. The result showed that the Service was deleted, but the Deployment and Pods were still running. This demonstrated that a Service is only responsible for exposing network access and does not delete the application Pods.

```
PS C:\Users\WangShiyu> kubectl delete service -l app=kubernetes-bootcamp
service "kubernetes-bootcamp" deleted
PS C:\Users\WangShiyu> kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
hello-minikube      NodePort    10.100.139.56 <none>         80:30523/TCP     17h
kubernetes          ClusterIP   10.96.0.1      <none>         443/TCP          21h
PS C:\Users\WangShiyu> kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
hello-minikube-75b65c4879-5wnjt     1/1     Running   0          17h
kubernetes-bootcamp-67fbdd6b79-m5fnx 1/1     Running   0          17h
```

6) Scale Your App

In this part, I scaled the kubernetes-bootcamp application to multiple replicas and then scaled it down.

The kubectl scale command modifies the spec.replicas field of a Deployment, triggering the Deployment controller to adjust the number of Pods.

Operation	Command	Replicas
Scale Up	<code>kubectl scale deployments/kubernetes-bootcamp --replicas=4</code>	4
Scale Down	<code>kubectl scale deployments/kubernetes-bootcamp --replicas=2</code>	2

- When scaling up: The Deployment creates new Pods using the Pod template defined in the Deployment specification
- When scaling down: The Deployment selects Pods to terminate based on a termination policy (typically preferring Pods with a NotReady status or those that have been running the shortest time)

(1) Recreate the Service

Since the Service had been deleted in the previous step, I recreated it first.

```
PS C:\Users\WangShiyu> kubectl expose deployment/kubernetes-bootcamp --type="NodePort" --port 8080
service/kubernetes-bootcamp exposed
PS C:\Users\WangShiyu> kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hello-minikube	NodePort	10.100.139.56	<none>	80:30523/TCP	17h
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	21h
kubernetes-bootcamp	NodePort	10.98.82.249	<none>	8080:31377/TCP	7s

(2) Check the current Deployment

```
PS C:\Users\WangShiyu> kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-minikube	1/1	1	1	18h
kubernetes-bootcamp	1/1	1	1	17h

(3) Scale the Deployment to 4 replicas

Scale-up process:

- The controller creates three new Pods
- Pods are scheduled onto available nodes (the single Minikube node in this case)
- The Scheduler considers resource requirements, node affinity, taints and tolerations, and other constraints when placing Pods

```
PS C:\Users\WangShiyu> kubectl scale deployments/kubernetes-bootcamp --replicas=4
deployment.apps/kubernetes-bootcamp scaled
```

(4) Check the Deployment again

The READY field showed that Kubernetes was creating more replicas.

```
PS C:\Users\WangShiyu> kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-minikube	1/1	1	1	18h
kubernetes-bootcamp	4/4	4	4	17h

(5) Check all Pods

The output showed that four kubernetes-bootcamp Pods were created and running. This means that Kubernetes successfully scaled the application horizontally.

```
PS C:\Users\WangShiyu> kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
hello-minikube-75b65c4879-5wnjt	1/1	Running	0	18h	10.244.0.16	minikube	<none>
kubernetes-bootcamp-67fbdd6b79-22gmq	1/1	Running	0	2m23s	10.244.0.18	minikube	<none>
kubernetes-bootcamp-67fbdd6b79-8ndtk	1/1	Running	0	2m23s	10.244.0.20	minikube	<none>
kubernetes-bootcamp-67fbdd6b79-m5dn8	1/1	Running	0	2m23s	10.244.0.19	minikube	<none>
kubernetes-bootcamp-67fbdd6b79-m5fnx	1/1	Running	0	17h	10.244.0.17	minikube	<none>

(6) observe the scaling events

The event records showed that the ReplicaSet had been scaled up.

```

PS C:\Users\WangShiyu> kubectl describe deployments/kubernetes-bootcamp
Name:                kubernetes-bootcamp
Namespace:           default
CreationTimestamp:   Thu, 07 May 2026 22:09:02 +0800
Labels:              app=kubernetes-bootcamp
Annotations:         deployment.kubernetes.io/revision: 1
Selector:             app=kubernetes-bootcamp
Replicas:            4 desired | 4 updated | 4 total | 4 available | 0 unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=kubernetes-bootcamp
  Containers:
    kubernetes-bootcamp:
      Image:      gcr.io/google-samples/kubernetes-bootcamp:v1
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:      <none>
      Volumes:     <none>
  Conditions:
    Type           Status  Reason
    ----           -
    Progressing    True    NewReplicaSetAvailable
    Available      True    MinimumReplicasAvailable
  OldReplicaSets:  <none>
  NewReplicaSet:   kubernetes-bootcamp-67fbdd6b79 (4/4 replicas created)
  Events:
    Type           Reason              Age             From              Message
    ----           -
    Normal         ScalingReplicaSet   5m33s          deployment-controller Scaled up replica set kubernetes-bootcamp-67fbdd6b79 from 1 to 4

```

(7) Access the application multiple times

In my Windows Docker driver environment, I used `minikube service kubernetes-bootcamp --url`, then I accessed the returned local URL several times.

```

PS C:\Users\WangShiyu> minikube service kubernetes-bootcamp --url
http://127.0.0.1:64507

```

The responses could come from different Pod names, showing that the Service can distribute requests among multiple Pods.

```

PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:64507
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-22gmq | v=1
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:64507
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-m5fnx | v=1
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:64507
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-22gmq | v=1
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:64507
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-22gmq | v=1
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:64507
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-m5dn8 | v=1

```

Kubernetes Services implement load balancing automatically using iptables or IPVS rules on each node. When a Service receives traffic, it distributes requests across all healthy Pods matching the Service selector.

The default load balancing algorithm is round-robin, though this can be configured. Each response includes the Pod name, allowing you to verify which Pod handled the request.

(8) Scale the Deployment down to 2 replicas and checked the Deployment and Pods

Scale-down process:

- The controller sends termination signals to selected Pods, allowing graceful shutdown
- The READY column shows x/y where x is the number of Pods passing readiness checks and y is the total desired count

Only two kubernetes-bootcamp Pods remained running. This showed that Kubernetes can automatically remove extra Pods when the replica number is reduced.

```

PS C:\Users\WangShiyu> kubectl scale deployments/kubernetes-bootcamp --replicas=2
deployment.apps/kubernetes-bootcamp scaled
PS C:\Users\WangShiyu> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
hello-minikube       1/1     1             1           18h
kubernetes-bootcamp  2/2     2             2           18h
PS C:\Users\WangShiyu> kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE
E READINESS GATES
hello-minikube-75b65c4879-5wnjt  1/1     Running   0          18h   10.244.0.16   minikube   <none>
kubernetes-bootcamp-67fbdd6b79-22gmq  1/1     Terminating  0      10m   10.244.0.18   minikube   <none>
kubernetes-bootcamp-67fbdd6b79-8ndtk  1/1     Running     0      10m   10.244.0.20   minikube   <none>
kubernetes-bootcamp-67fbdd6b79-m5dn8  1/1     Terminating  0      10m   10.244.0.19   minikube   <none>
kubernetes-bootcamp-67fbdd6b79-m5fnx  1/1     Running     0      18h   10.244.0.17   minikube   <none>

```

7) Update Your App

Kubernetes uses a rolling update strategy to update applications without downtime.



Rolling updates are a core feature of Kubernetes Deployments that enable zero-downtime deployments.

- The Deployment controller creates a new ReplicaSet for the updated version while maintaining the old ReplicaSet
- During the transition, both versions run simultaneously, ensuring continuous availability
- Pods running the old version are gradually replaced with Pods running the new version

(1) Check the current image version

The original image version was the bootcamp application image, through using `kubectl describe pods`.

```

Containers:
  kubernetes-bootcamp:
    Container ID:   docker://ad6ff119592a86775b4fdc890b205f6f81d4b5d1c21d856d08224ff58c667766
    Image:          gcr.io/google-samples/kubernetes-bootcamp:v1
    Image ID:       docker-pullable://gcr.io/google-samples/kubernetes-bootcamp@sha256:0d6b8ee63bb57c5f5b6156f446b3b

```

(2) Update the Deployment to version 2

In my actual experiment environment, the Google Container Registry image could not always be pulled successfully due to network access limitations. Therefore, I also used the Docker Hub mirror image and checked the rollout status.

The output showed that the Deployment was successfully rolled out.

```

PS C:\Users\WangShiyu> kubectl set image deployment/kubernetes-bootcamp kubernetes-bootcamp=docker.io/jocatalin/kubern
es-bootcamp:v2
PS C:\Users\WangShiyu> kubectl rollout status deployment/kubernetes-bootcamp
deployment "kubernetes-bootcamp" successfully rolled out

```

- `kubectl set image`: Updates the container image specified in the Deployment's Pod template

- The container name `kubernetes-bootcamp` must match the container name defined in the Deployment, which can be verified using `kubectl describe deployment`
- rollout status: Monitors the progress of a deployment update and exits when the update completes successfully

(3) Check the Deployment image

Infrastructure layer: The `kubectl get deployment -o wide` command shows the current image specification, confirming the configuration was updated.

The image was shown as: `docker.io/jocatalin/kubernetes-bootcamp:v2`

```
PS C:\Users\WangShiyu> kubectl get deployment kubernetes-bootcamp -o wide
```

NAME	SELECTOR	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES
kubernetes-bootcamp		2/2	2	2	22h	kubernetes-bootcamp	docker.io/jocatalin/kubernetes-bootcamp:v2

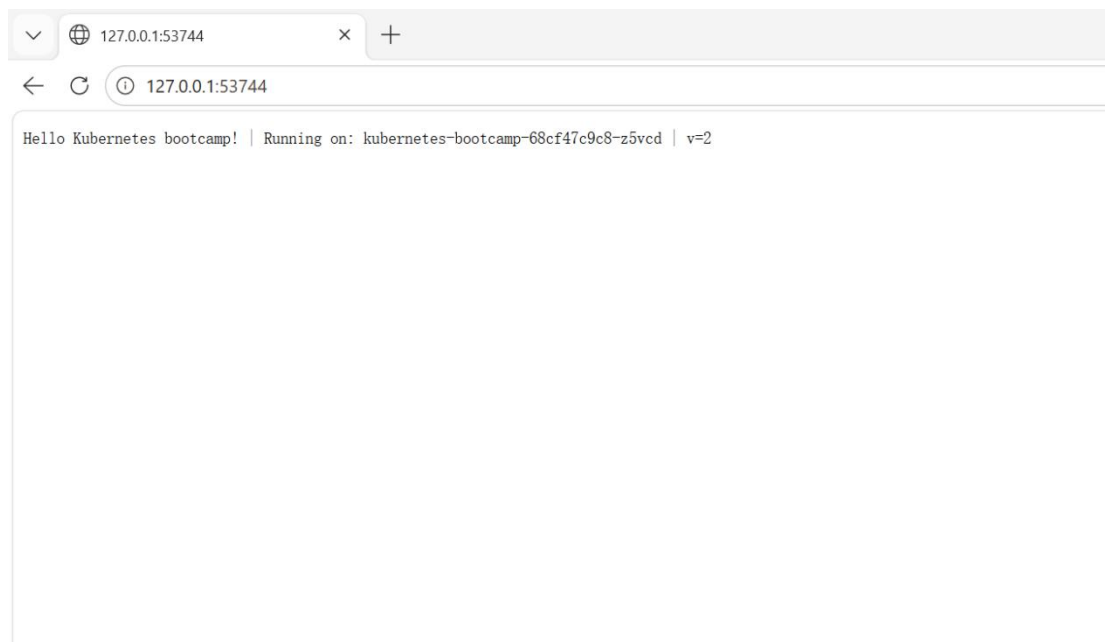
(4) Access the application through the Minikube service URL

Application layer: The `--url` command tests the actual application behavior, verifying that the running containers are using the new image

```
PS C:\Users\WangShiyu> minikube service kubernetes-bootcamp --url http://127.0.0.1:53744
```

The output was:

```
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:53744
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-68cf47c9c8-z5vcd | v=2
```



This proved that the application had been successfully updated to version 2.

(5) Simulate an update failure

I intentionally updated the application to a nonexistent version `v10`.

```
PS C:\Users\WangShiyu> kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=gcr.io/google-samples/kubernetes-bootcamp:v10
deployment.apps/kubernetes-bootcamp image updated
```

(6) Check the Pod status

A new Pod appeared with the status: `ErrImagePull`, which means Kubernetes tried to pull the

nonexistent or unavailable image but failed.

```
PS C:\Users\WangShiyu> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
hello-minikube-75b65c4879-5wnjt    1/1     Running   2 (68m ago) 23h
kubernetes-bootcamp-68cf47c9c8-xkvqb 1/1     Running   0           32m
kubernetes-bootcamp-68cf47c9c8-z5vcd 1/1     Running   0           20m
kubernetes-bootcamp-845bc89d75-fr26s 0/1     ErrImagePull 0           25s
```

- ErrImagePull: The first failure attempt when Kubernetes tries to pull the image

(7) Roll back the Deployment and check the rollout status

```
PS C:\Users\WangShiyu> kubectl rollout undo deployments/kubernetes-bootcamp
deployment.apps/kubernetes-bootcamp rolled back
PS C:\Users\WangShiyu> kubectl rollout status deployments/kubernetes-bootcamp
deployment "kubernetes-bootcamp" successfully rolled out
```

- rollout undo: Reverts a Deployment to its previous revision

(8) Check the Pods again

The result showed that the failed Pod was removed and the remaining kubernetes-bootcamp Pods were in the Running state.

```
PS C:\Users\WangShiyu> kubectl rollout undo deployments/kubernetes-bootcamp
deployment.apps/kubernetes-bootcamp rolled back
PS C:\Users\WangShiyu> kubectl rollout status deployments/kubernetes-bootcamp
deployment "kubernetes-bootcamp" successfully rolled out
PS C:\Users\WangShiyu> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
hello-minikube-75b65c4879-5wnjt    1/1     Running   2 (74m ago) 23h
kubernetes-bootcamp-68cf47c9c8-xkvqb 1/1     Running   0           38m
kubernetes-bootcamp-68cf47c9c8-z5vcd 1/1     Running   0           26m
PS C:\Users\WangShiyu> minikube service kubernetes-bootcamp --url
http://127.0.0.1:51910
```

(9) Access the application again

The application responded normally and showed version v=2, which proved that the rollback was successful.

```
PS C:\Users\WangShiyu> curl.exe http://127.0.0.1:51910
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-68cf47c9c8-z5vcd | v=2
```

8) Resource Cleanup

Resource cleanup is an essential practice in Kubernetes management to avoid unnecessary resource consumption and cost.

After finishing all experiments, I cleaned up the Kubernetes resources.

(1) Delete the Deployments and Services

```
PS C:\Users\WangShiyu> kubectl delete deployment hello-minikube
deployment.apps "hello-minikube" deleted
PS C:\Users\WangShiyu> kubectl delete service hello-minikube
service "hello-minikube" deleted
PS C:\Users\WangShiyu> kubectl delete deployment kubernetes-bootcamp
deployment.apps "kubernetes-bootcamp" deleted
PS C:\Users\WangShiyu> kubectl delete service kubernetes-bootcamp
service "kubernetes-bootcamp" deleted
```

(2) Check whether the resources had been removed

```
PS C:\Users\WangShiyu> kubectl get all
NAME                                TYPE           CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/kubernetes                  ClusterIP      10.96.0.1    <none>        443/TCP    27h
```

- kubectl get all: Lists all resources in the default namespace

(3) Stop the Minikube cluster

```
PS C:\Users\WangShiyu> minikube stop
* 正在停止节点 "minikube" ...
* 正在通过 SSH 关闭 "minikube"...
* 1 个节点已停止。
```

- minikube stop:
 - ◆ Gracefully shuts down the Minikube cluster while preserving the cluster state and data
 - ◆ Useful for saving system resources when not actively working with the cluster
 - ◆ Allows quick restart later with minikube start

3. Conclusion

This experiment successfully provided a comprehensive, hands-on introduction to Kubernetes, covering the entire application lifecycle from cluster setup to deployment, scaling, and updating. Through the completion of the Minikube tutorials, theoretical knowledge was effectively translated into practical operational skills.

1) Summary of Completed Tasks

Phase	Tasks Completed	Status
Cluster Setup	Deployed a local single-node Kubernetes cluster using Minikube with Docker driver; verified connectivity using kubectl.	Completed
Application Deployment	Created a Deployment using a sample container image; verified Pod scheduling and status.	Completed
Application Exploration	Inspected Pod details, viewed container logs, and executed commands inside running containers using kubectl exec.	Completed
Service Exposure	Exposed the application using a NodePort Service; verified external access through the Minikube URL.	Completed
Application Scaling	Scaled the Deployment from 1 to 4 replicas; verified that the Service distributed traffic across all Pods.	Completed
Update and Rollback	Performed a rolling update to a new image version; successfully executed a rollback to the previous version.	Completed

2) Challenges and Solutions

(1) Image Pull Latency

Description: Initial Pod startup was slow due to container image download time.

Resolution: Used `kubectl get pods -w` to monitor status changes in real-time.

(2) Service Access

Description: NodePort Service required special tunneling in the local Minikube environment.

Resolution: Used `minikube service <name>` command to generate an accessible URL.

In conclusion, this experiment successfully achieved its objective of establishing a foundational understanding of Kubernetes orchestration. It validated the core principles of container management: abstraction through Services, resilience through Controllers, and agility through Rolling Updates.